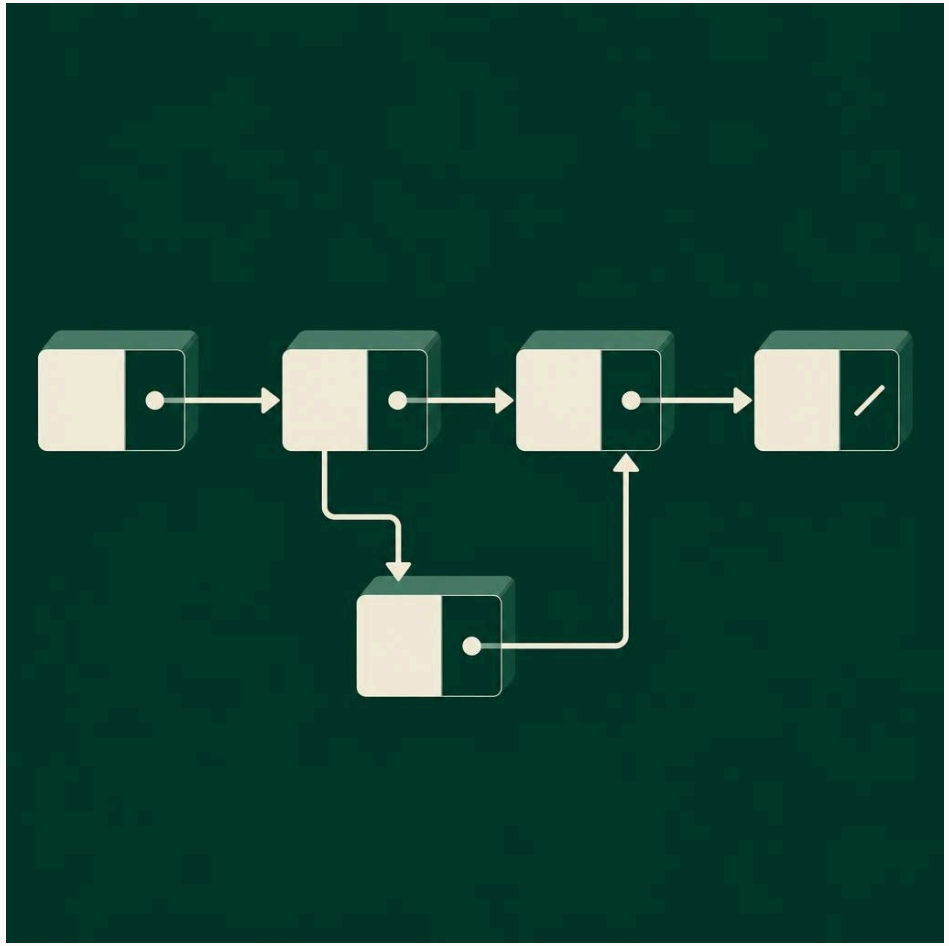
malloc e free in C

Allocare e liberare, funzioni e memoria dinamica, errori con la memoria

LINGUAGGIO C | MEMORIA DINAMICA | MALLOC E FREE



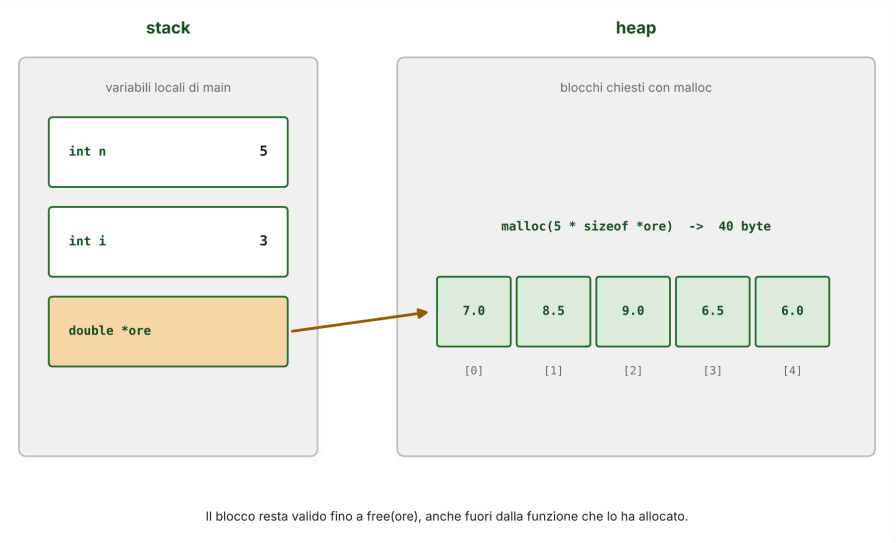
Stack e Heap

Le variabili locali stanno nello stack: la loro dimensione è fissata quando si compila e spariscono all'uscita dal blocco. Lo heap è la memoria che il programma chiede mentre è in esecuzione.

```c
#include <stdlib.h>

double *ore = malloc(n * sizeof *ore);   /* n double */

if (ore == NULL) {
    printf("Memoria insufficiente.\n");
} else {
    ore[0] = 7.0;                 /* si usa come un array */
    ...
    free(ore);                    /* il blocco torna al sistema */
}
```



malloc riceve i byte e restituisce l'indirizzo del blocco, oppure NULL. Il contenuto non è inizializzato. Ogni blocco si libera con free, una sola volta.

Un Array della Dimensione Giusta

```
printf("Quanti giorni? ");
scanf("%d", &n);

ore = malloc(n * sizeof *ore);

printf("%zu byte\n", n * sizeof *ore);  /* 40 */
printf("%zu byte\n", sizeof ore);       /* 8: il puntatore */

for (i = 0; i < n; i++) {
    totale = totale + ore[i];
}
```

	Array locale	**Blocco con malloc**
dimensione	fissata quando si compila	scelta durante l'esecuzione
durata	fino alla fine del blocco	fino alla free
sizeof	dimensione dell'array	dimensione del puntatore
errore di memoria	il programma termina	malloc restituisce NULL

> sizeof *ore segue il tipo del puntatore: se ore diventa un float *, la malloc resta corretta. Il numero di elementi va tenuto in una variabile.

Struct nello Heap

```
Studente *eletto = malloc(sizeof *eletto);
Studente *candidati = malloc(n * sizeof *candidati);

strcpy(candidati[i].nome, "Sara Conti");
candidati[i].voti = 52;        /* elemento: punto */

*eletto = candidati[migliore];
printf("%s", eletto->nome);      /* puntatore: freccia */

free(candidati);
free(eletto);
```

Una struct o un array di struct si allocano con una sola malloc, e una sola free restituisce tutto il blocco, compresi gli array che la struct contiene.

- eletto è un puntatore: i campi si raggiungono con la freccia
- candidati[i] è una struct: i campi si raggiungono con il punto

> free(NULL) non fa nulla: le due free sono corrette anche se una sola delle due malloc è riuscita.

Funzioni che Allocano

```
int *crea_tabellina(int numero, int n)
{
   int *v = malloc(n * sizeof *v);
   int i;

   if (v != NULL) {
      for (i = 0; i < n; i++) {
         v[i] = numero * (i + 1);
      }
   }
   return v;
}

int *t = crea_tabellina(7, 10);
...
free(t);
```

Un blocco dello heap non appartiene a una funzione: resta valido fino alla free, anche dopo che la funzione è terminata.

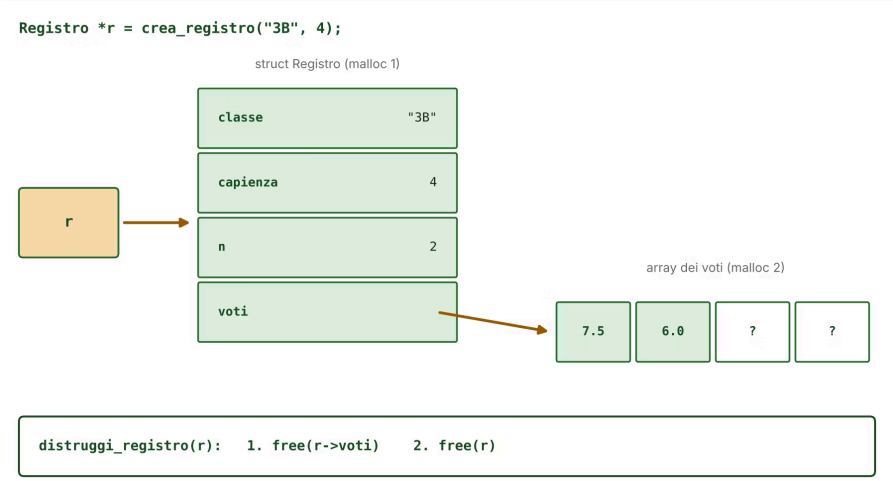
Chi fornisce la memoria	Prototipo
il chiamante	void scrivi(char testo[], ...)
la funzione	char *crea(...), libera il chiamante

> Mai restituire l'indirizzo di un array locale: sparisce con la funzione e il chiamante riceve un puntatore pendente.

Coppie crea e distruggi

Se una struct contiene un puntatore a un altro blocco, le malloc stanno in crea_... e le free in distruggi_...: chi usa il tipo non chiama mai malloc o free sui suoi pezzi.

```c
Registro *crea_registro(const char *classe, int capienza)
{
    Registro *r = malloc(sizeof *r);
    if (r != NULL) {
        r->voti = malloc(capienza * sizeof *r->voti);
        if (r->voti == NULL) {
            free(r);            /* niente perdite */
            r = NULL;
        }
    }
    return r;
}
```



> distruggi_registro: prima free(r->voti), poi free(r).

Lunghezza in Uscita e Matrici in un Blocco

```
double *filtra(const double misure[], int n,
               double minimo, double massimo,
               int *quante);

valide = filtra(letture, n, 300, 5000, &n_valide);

double *tab = malloc(righe * colonne * sizeof *tab);

tab[r * colonne + c]        /* elemento (r, c) */
```

Un blocco non dice quanto è lungo: una funzione che restituisce un array nuovo comunica la lunghezza con un parametro di uscita.

filtra fa due passate: conta gli elementi da tenere, alloca il blocco giusto, poi li copia.

> Una matrice di dimensioni scelte all'avvio sta in un solo blocco, con le righe una dopo l'altra: una malloc e una free.

Errori con la Memoria

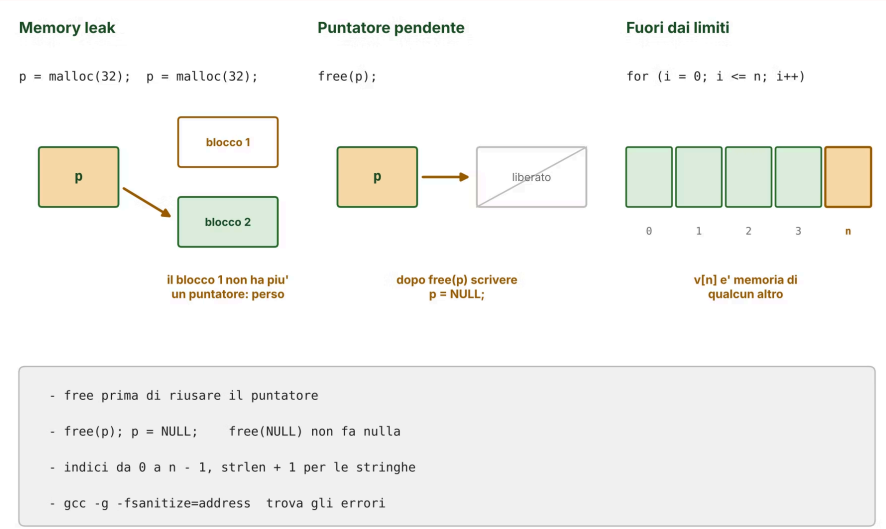
Il compilatore non controlla come si usa lo heap: questi errori compilano senza avvisi e spesso si manifestano lontano da dove sono stati commessi.

```
avviso = malloc(LUNG);
avviso = malloc(LUNG);     /* memory leak */

free(p);
p->ora = 3;              /* uso dopo free */
free(p);                 /* doppio free */

for (i = 0; i <= n; i++)   /* v[n] è fuori dal blocco */

free(p);
p = NULL;                /* corretto */
```



> gcc -g -fsanitize=address ferma il programma al primo accesso sbagliato e indica la riga.

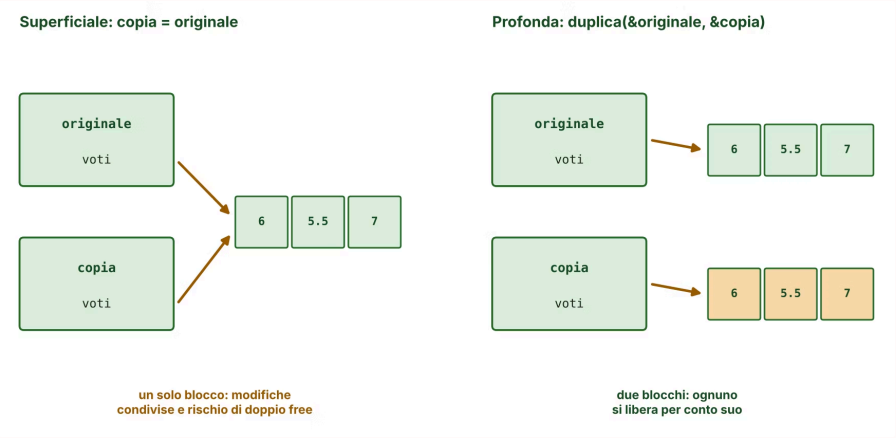
Copia Superficiale e Copia Profonda

L'assegnamento tra struct copia i campi: se un campo è un puntatore, la copia punta allo stesso blocco.

```
typedef struct { char classe[4]; int n; double *voti; } Registro;

Registro prova = originale;     /* stesso array dei voti */

int duplica(const Registro *da, Registro *a)
{
   *a = *da;
   a->voti = malloc(da->n * sizeof *a->voti);
   ...copia i voti...
}
```



Con la copia superficiale le modifiche arrivano all'originale e il blocco va liberato una volta sola; la copia profonda ha un blocco suo.

Righe Sicure e Array di Puntatori

```
char buffer[32];
char *bacheca[MAX];

fgets(buffer, 32, stdin);      /* mai più di 31 caratteri */
...
copia = malloc(strlen(buffer) + 1);
strcpy(copia, buffer);
bacheca[n] = copia;

free(scaffale[k]);
scaffale[k] = NULL;          /* scaffale libero */
```

fgets non supera mai la dimensione del buffer; se la riga è più lunga il resto va consumato con getchar. Ogni testo accettato va in un blocco di strlen + 1 byte.

In un array di puntatori NULL indica una posizione libera: si riusa, un secondo free diventa free(NULL), alla fine si liberano i puntatori non NULL.

> Due contatori, allocati e liberati, controllano a fine programma che nessun blocco sia rimasto in memoria.

Errori Comuni da Evitare

1

Usare il risultato di malloc senza controllare NULL.

2

Calcolare i byte con il tipo sbagliato invece di sizeof *p.

3

Contare gli elementi con sizeof sul puntatore.

4

Allocare strlen byte dimenticando il '\0'.

5

Restituire l'indirizzo di un array locale.

6

Liberare la struct prima del blocco che contiene.

7

Riusare un puntatore senza free: il vecchio blocco è perso.

8

Usare o liberare di nuovo un puntatore dopo free.

Riepilogo Concetti

Situazione	Tecnica	Note
Dimensione scelta durante l'esecuzione	p = malloc(n * sizeof *p)	controllare NULL
Restituire la memoria	free(p)	una volta sola, free(NULL) innocuo
Struct nello heap	s->campo, v[i].campo	una free per blocco
Funzione che crea un array	int *crea(int n)	libera il chiamante
Lunghezza di un risultato	parametro int *quante	il blocco non la ricorda
Struct con puntatori	crea_... e distruggi_...	prima i blocchi interni
Matrice di dimensioni variabili	tab[r * colonne + c]	un solo blocco
Dopo free	p = NULL	niente puntatori pendenti
Struct con puntatore da copiare	copia profonda	un blocco per copia
Trovare gli errori	-fsanitize=address	indica la riga

SEZIONE 1

Esempi Pratici: Allocare e Liberare

Concetti base — tre file su malloc, array e struct.

1 – 010-prima-malloc.c

Problema

un contatore nello stack e uno nello heap.

Tecnica

malloc(sizeof *p), controllo di NULL, free.

2 – 020-array-di-dimensione-scelta.c

Problema

le ore di sonno di un periodo scelto.

Tecnica

malloc(n * sizeof *ore) e sizeof del puntatore.

3 – 030-struct-allocata.c

Problema

l'elezione del rappresentante di classe.

Tecnica

struct e array di struct nello heap.

SEZIONE 1

Esempi Pratici: Problemi su Allocare e Liberare

040 – Voti della verifica

Classe di n studenti: media, voti sopra la media, insufficienze.

050 – Misure del laboratorio

Periodo del pendolo: media, scarto e misure anomale.

060 – Scheda dello studente

Una struct allocata per riga e riempita da una funzione.

070 – Posti della gita

Array di partecipanti con la capienza del pullman.

080 – Copia di una stringa

Nomi dei diplomati in blocchi di strlen + 1 byte.

SEZIONE 2

Esempi Pratici: Funzioni e Memoria Dinamica

Concetti base — tre file su funzioni, buffer e proprietà.

1 – 010-funzione-che-alloca.c

Problema

le tabelline per il ripasso.

Tecnica

crea_tabellina restituisce il blocco.

2 – 020-non-restituire-un-locale.c

Problema

le etichette dei banchi del laboratorio.

Tecnica

buffer del chiamante o blocco allocato.

3 – 030-chi-libera.c

Problema

i registri di due classi.

Tecnica

crea_registro e distruggi_registro.

SEZIONE 2

Esempi Pratici: Problemi su Funzioni e Memoria Dinamica

040 – Unire stringhe

Indirizzi email della classe nella lunghezza esatta.

050 – Tabella dei voti

Tabellone dello scrutinio in un solo blocco.

060 – Filtrare le misure

Letture valide del sensore di CO2 in un array nuovo.

070 – Elenco degli iscritti

Corsi pomeridiani con crea, iscrivi, ritira e distruggi.

080 – Torneo

Squadre nello heap e classifica con indici temporanei.

SEZIONE 3

Esempi Pratici: Errori con la Memoria

Concetti base — tre file su perdite, free e limiti.

1 – 010-memory-leak.c

Problema

gli avvisi sul monitor dell'atrio.

Tecnica

contatori di blocchi allocati e liberati.

2 – 020-doppio-free-e-puntatore-pendente.c

Problema

una prenotazione annullata due volte.

Tecnica

libera(&p) con free e p = NULL.

3 – 030-fuori-dai-limiti.c

Problema

i computer dell'aula studio.

Tecnica

un ciclo con <= rovina l'array vicino.

SEZIONE 3

Esempi Pratici: Problemi su Errori con la Memoria

040 – Contatore delle allocazioni

Registro dei blocchi: perdite e doppi free segnalati.

050 – Riga sicura

Messaggi della bacheca senza overflow.

060 – Prestiti dell'attrezzatura

Array di puntatori con NULL per lo scaffale libero.

070 – Copia del registro

Copia superficiale e profonda prima dello scrutinio.

080 – Archivio delle foto

Foto e didascalie: memoria su due livelli.